

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Pertemuan 10 Testing, DI, dan Debugging

Menjaga fitur lama tetap benar dengan unit test, UI test, dan dependensi yang disusun Koin

Muhammad Habib Algifari, S.Kom., M.T.I.

Tahun 2026



Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Tiga jawaban dari tiket keluar minggu lalu

Pertanyaan	Jawaban singkat
Kenapa kunci API tidak boleh ada di repository?	Repository bisa dibaca orang lain, dan kunci bisa disalahgunakan sampai kuota habis
Apa gunanya meminta keluaran JSON dengan skema?	Hasilnya bisa langsung di-parse ke data class dan dicek
Kenapa hasil AI ditampilkan sebagai saran yang bisa dikoreksi?	AI bisa salah, keputusan tetap di pengguna

Fitur lama yang diam-diam rusak

Setiap minggu kita menambah fitur. Yang dicoba hanya fitur barunya, karena mengetes ulang lima layar satu per satu terlalu lama. Riwayat commit LaporKampus di samping bisa terjadi di kelompok mana saja.

DISKUSI 2 MENIT

Bagaimana memastikan kode tetap benar setiap kali menambah fitur, tanpa mengetes manual satu per satu?

Minggu 6

Form mengirim laporan ke server.

Dicoba manual, aman

Minggu 9

Tombol Pakai saran AI. Di commit yang sama `isNotBlank` di `bisaKirim` ikut berubah menjadi `isEmpty`.

Form tidak dicoba ulang

Demo

Laporan berjudul tiga spasi masuk ke admin fasilitas.

Ketahuan di depan penguji

Saat pulang nanti, kalian bisa

1

Menulis unit test

Menguji aturan UiState dan repository dengan kotlin.test dan fake.

2

Menguji ViewModel dan layar

Memakai runTest untuk ViewModel dan runComposeUiTest untuk layar.

3

Menyusun dependensi

Merakit sumber data dan ViewModel dengan Koin, lalu menggantinya saat test.

Capaian ini mendukung **CPMK0502**: mahasiswa mampu menjelaskan konsep pemrograman untuk pengembangan perangkat lunak, dalam bentuk test yang menjelaskan perilaku kode dan dependensi yang tersusun rapi.

Bagian 1

Unit test pertama

Kenapa test, di mana menaruhnya, dan fake yang menggantikan server.

Piramida test

UI test

Sedikit, lebih lambat

Integration test

Beberapa, lintas lapisan, minggu 13

Unit test

Banyak, cepat, satu kelas atau fungsi

Makin ke bawah, makin banyak, makin cepat, dan makin mudah menunjuk baris yang salah.

Bank Proyek meminta minimal 10 unit test dan 3 UI test. Integration test kita bahas di minggu 13.

Test tinggal di commonTest

composeApp/build.gradle.kts

```
commonTest.dependencies {  
    implementation(kotlin("test"))  
    implementation(libs.kotlinx.coroutines.test)  
    implementation(libs.koin.test)  
    // versi sama dengan Compose Multiplatform  
    implementation(libs.compose.ui.test)  
}  
desktopTest.dependencies {  
    implementation(compose.desktop.currentOs)  
}
```

Folder	Isinya
commonMain	Kode aplikasi
commonTest	Test untuk semua platform
androidUnitTest	Test yang butuh API Android

Jalankan dengan `./gradlew :composeApp:desktopTest`.
Tidak butuh emulator.

Test pertama: Arrange, Act, Assert

commonTest/ui/FormLaporanUiStateTest.kt

```
class FormLaporanUiStateTest {  
    @Test  
    fun formKosongTidakBisaDikirim() {  
        val state = FormLaporanUiState() // Arrange  
        val hasil = state.bisaKirim // Act  
        assertFalse(hasil) // Assert  
    }  
  
    @Test  
    fun judulHanyaSpasiTidakBisaDikirim() {  
        val state = FormLaporanUiState(  
            judul = " ", lokasi = "GKU 1 R201")  
        assertFalse(state.bisaKirim)  
    }  
}
```

Bagian	Artinya
@Test	Fungsi ini dijalankan sebagai test
assertFalse	Gagal kalau nilainya true
assertEquals	Gagal kalau dua nilai berbeda

Nama test adalah kalimat perilaku. Saat merah, namanya langsung memberi tahu aturan mana yang rusak.

BANGUN BARENG

Di commit minggu 9, bisaKirim memakai judul.isNotEmpty()

Kita jalankan empat test bisaKirim. Berapa yang lulus, dan test mana yang gagal? Tulis tebakanmu, baru kita jalankan.

Cek tebakanmu: dari merah ke hijau

Test merah menjadi hijau

FormLaporanUiState.kt

```
val bisaKirim: Boolean
    get() =
        judul.isNotBlank() &&
        lokasi.isNotBlank() &&
        !sedangMengirim
```

Satu kata diganti, lalu test dijalankan lagi.

4 lulus, 0 gagal

desktopTest

LULUS formKosongTidakBisaDikirim

LULUS judulDanLokasiTerisiBisaDikirim

LULUS judulHanyaSpasiTidakBisaDikirim

LULUS sedangMengirimTidakBisaLagi

Semua hijau. Test yang sama akan menjaga aturan ini di setiap perubahan.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

COBA LANGSUNG



Scan atau klik, tekan Run. Satu test gagal. Ganti isEmpty menjadi isNotBlank, lalu Run lagi.

[Jalankan test bisaKirim di Kotlin Playground](#)

Sumber: Kotlin Playground, JetBrains. Fungsi cek buatan sendiri menggantikan assert dari kotlin.test.

Fake: Sumber tiruan untuk test

commonTest/data/SumberUji.kt

```
class SumberUji : Sumber<Laporan> {  
    val terkirim = mutableListOf<Laporan>()  
    var gagal = false  
  
    override suspend fun semua() = terkirim.toList()  
    override suspend fun ambil(id: Int) =  
        terkirim.first { it.id == id }  
    override fun pantau(id: Int) =  
        flowOf(terkirim.first { it.id == id })  
    override suspend fun kirim(baruk: Laporan): Laporan {  
        if (gagal) error("internet kampus mati")  
        val hasil = baru.copy(id = terkirim.size + 1)  
        terkirim += hasil  
        return hasil  
    }  
}
```

Sifat	Di SumberUji
Cepat	Tanpa jaringan dan tanpa delay
Bisa diatur	gagal = true
Bisa diperiksa	terkirim.size

Polanya sama dengan `SumberLaporanPalsu` dan `AsistenPalsu`. Bedanya, yang ini dibuat khusus untuk diperiksa.

Bagian 2

Menguji ViewModel dan layar

`runTest` untuk coroutine di ViewModel, dan `runComposeUiTest` untuk layar.

Test FormLaporanViewModel

commonTest/ui/FormLaporanViewModelTest.kt

```
class FormLaporanViewModelTest {  
    private val sumber = SumberUji()  
    @BeforeTest fun mulai() =  
        Dispatchers.setMain(StandardTestDispatcher())  
    @AfterTest fun selesai() = Dispatchers.resetMain()  
    @Test  
    fun kirimBerhasilMengosongkanForm() = runTest {  
        val vm = FormLaporanViewModel(sumber)  
        vm.ubahJudul("AC mati")  
        vm.ubahLokasi("GKU 1 R201")  
        vm.kirim()  
        advanceUntilIdle()  
        assertEquals(1, sumber.terkirim.size)  
        assertEquals("", vm.uiState.value.judul)  
    }  
}
```

Bagian	Gunanya
setMain	viewModelScope memakai dispatcher test
runTest	Menjalankan test coroutine dengan waktu virtual
advanceUntilIdle	Jalankan semua coroutine yang antre

ViewModel adalah kelas biasa. Cukup panggil fungsinya, lalu periksa state.

BANGUN BARENG

Baris `advanceUntilIdle()` kita hapus

Test `kirimBerhasilMengosongkanForm` dijalankan lagi. Lulus atau gagal? Kalau gagal, assert yang mana dan apa pesannya?

Menguji AsistenViewModel saat gagal

commonTest/ui/AsistenViewModelTest.kt

```
class AsistenGagal : AsistenLaporan {  
    override suspend fun sarankan(isi: String) =  
        Result.failure<SaranLaporan>(  
            IllegalStateException("kuota habis"))  
}  
  
@Test  
fun asistenGagalMenampilkanPesan() = runTest {  
    val vm = AsistenViewModel(AsistenGagal())  
    vm.minta("AC di GKU 1 R201 netes ke meja")  
    advanceUntilIdle()  
    assertIs<AsistenUiState.Gagal>(vm.ui.value)  
}
```

Test	Fake	Diharapkan
Awal	Apa saja	Kosong
Berhasil	AsistenPalsu	Saran
Gagal	AsistenGagal	Gagal

`delay(1000)` di `AsistenPalsu` tidak membuat test menunggu satu detik, karena `runTest` memakai waktu virtual.

UI test dengan runComposeUiTest

commonTest/ui/FormLaporanUiTest.kt

```
@OptIn(ExperimentalTestApi::class)
class FormLaporanUiTest {
    @Test
    fun kirimAktifSetelahFormDiisi() = runComposeUiTest {
        setContent {
            FormLaporanScreen(
                vm = FormLaporanViewModel(SumberUji())
            )
            onNodeWithText("Kirim").assertIsNotEnabled()
            onNodeWithText("Judul").performTextInput("AC mati")
            onNodeWithText("Lokasi")
                .performTextInput("GKU 1 R201")
            onNodeWithText("Kirim").assertIsEnabled()
        }
    }
}
```

Layar diberi ViewModel dengan `SumberUji`. Tidak ada jaringan, dan tidak butuh Koin.

Composable bisa diuji, tetapi butuh layar yang menyala. Karena itu logika ditaruh di ViewModel, dan UI test cukup memeriksa yang tampil.

Cari, lakukan, periksa

Langkah	Fungsi	Contoh
Cari node	<code>onNodeWithText</code> , <code>onNodeWithTag</code>	<code>onNodeWithTag("tombolKirim")</code>
Lakukan aksi	<code>performClick</code> , <code>performTextInput</code>	<code>performTextInput("AC mati")</code>
Periksa	<code>assertIsDisplayed</code> , <code>assertIsEnabled</code>	<code>assertTextEquals("Laporan terkirim")</code>
Pastikan hilang	<code>assertDoesNotExist</code>	Kartu Saran AI setelah Abaikan

Teks tombol bisa berubah. Untuk node penting, beri `Modifier.testTag("tombolKirim")` lalu cari dengan `onNodeWithTag`, seperti contoh di dokumentasi Compose Multiplatform.

Bagian 3

Dependency injection dengan Koin

Siapa yang membuat objek, dan cara menggantinya saat test.

Merakit dependensi dengan tangan

commonMain/App.kt

```
@Composable
fun LaporkampusApp() {
    val client = remember { buatClient() }
    val sumber = remember {
        SumberLaporanJaringan(LaporanApi(client))
    }
    NavHost(nav, startDestination = RuteBeranda) {
        composable<RuteForm> {
            val vm = viewModel { FormLaporanViewModel(sumber) }
            FormLaporanScreen(vm)
        }
        // Detail, Asisten, Profil: dirakit dengan cara sama
    }
}
```

Dependency injection

Objek menerima dependensinya dari luar, lewat konstruktor, bukan membuatnya sendiri. Kelas kita sudah begitu sejak minggu 5.

Yang berantakan adalah tempat merakitnya. App.kt tahu semua kelas, dari HttpClient sampai setiap ViewModel.

Modul Koin untuk Laporan Kampus

commonMain/di/ModulAplikasi.kt

```
val dataModule = module {
    single { buatClient() }
    single { LaporanApi(get()) }
    single<Sumber<Laporan>> { SumberLaporanJaringan(get()) }
    single<AsistenLaporan> { AsistenPalsu() }
}

val uiModule = module {
    viewModel { FormLaporanViewModel(get()) }
    viewModel { AsistenViewModel(get()) }
    viewModel { (id: Int) → DetailViewModel(id, get()) }
}
```

Kata	Artinya
single	Satu objek untuk seluruh aplikasi
factory	Objek baru setiap kali diminta
viewModel	ViewModel yang ikut umur layar
get()	Koin mencari objek bertipe yang dibutuhkan

BANGUN BARENG

Sumber kita ubah dari single menjadi factory

Pelapor mengirim laporan dari form, lalu membuka detailnya. Apakah laporan baru itu ditemukan? Tulis tebakanmu, baru kita coba dengan SumberLaporanPalsu.

Memasang Koin di aplikasi

commonMain/App.kt

```
@Composable
fun App() {
    KoinApplication(application = {
        modules(dataModule, uiModule)
    }) {
        LaporKampusApp()
    }
}
```

commonMain/ui/FormLaporanScreen.kt

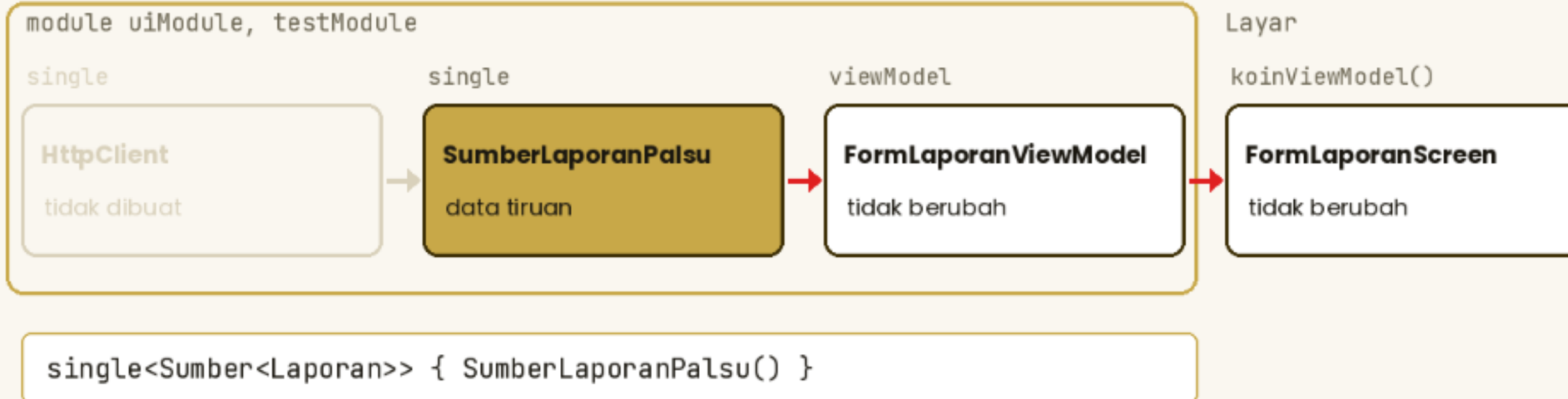
```
@Composable
fun FormLaporanScreen(
    vm: FormLaporanViewModel = koinViewModel()
) { /* isi sama seperti minggu 4 */ }
```

Library	Isinya
koin-core	module, single, factory, get
koin-compose	KoinApplication
koin-compose-viewmodel	viewModel dan koinViewModel
koin-test	KoinTest untuk test modul

Isi layar tidak berubah. Yang berubah hanya cara memperoleh ViewModel.

Graf dependensi yang disusun Koin

Saat test: Koin memakai modul fake



Hanya satu definisi diganti. ViewModel dan layar tetap sama persis.

COBA LANGSUNG



Scan atau klik untuk membaca cara memasang Koin di Compose Multiplatform, termasuk KoinApplication dan koinViewModel.

[Koin for Compose, insert-koin.io](https://insert-koin.io)

Sumber: Dokumentasi resmi Koin. Untuk test: [Koin Testing](#), tautan dari README hands-on P11.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Modul uji menggantikan modul data

commonTest/di/ModulAplikasiTest.kt

```
val modulUji = module {
    single<Sumber<Laporan>> { SumberUji() }
    single<AsistenLaporan> { AsistenGagal() }
}

class ModulAplikasiTest : KoinTest {
    @BeforeTest fun mulai() {
        startKoin { modules(uiModule, modulUji) }
    }
    @AfterTest fun selesai() = stopKoin()

    @Test
    fun formViewModelDirakitKoin() {
        val vm = get<FormLaporanViewModel>()
        assertFalse(vm.uiState.value.bisaKirim)
    }
}
```

`uiModule` sama persis dengan aplikasi. Hanya modul data yang diganti.

`stopKoin()` wajib di `AfterTest`. Tanpa itu, test berikutnya gagal dengan `KoinAppAlreadyStartedException`.

Bagian 4

Debugging

Membaca stack trace dan menyaring Logcat, untuk crash yang lolos dari test.

Kode yang akan crash

File.kt, di Kotlin Playground

```
data class LaporanDto(val id: Int, val prioritas: String)
data class Laporan(val id: Int, val prioritas: Int)

fun LaporanDto.keLaporan() = Laporan(id, prioritas.toInt())

fun muat(dto: LaporanDto): Laporan = try {
    dto.keLaporan()
} catch (e: NumberFormatException) {
    throw IllegalStateException("Gagal memuat ${dto.id}", e)
}

fun main() {
    val dariServer = listOf(LaporanDto(1, "2"),
        LaporanDto(7, "tinggi"), LaporanDto(9, "1"))
    for (dto in dariServer) println("Siap: ${muat(dto)}")
}
```

Server mengirim prioritas sebagai teks. Biasanya isinya angka, tetapi satu laporan berisi "tinggi".

muat menangkap error itu lalu melempar error baru dengan pesan yang lebih jelas, sambil membawa error aslinya.

BANGUN BARENG

Kode ini kita jalankan

Apa yang tercetak sebelum program berhenti? Lalu, baris nomor berapa di File.kt yang harus diubah supaya ketiga laporan tercetak?

Cek tebakanmu: membaca keluarannya

Keluaran, baris panjang dipecah

```
Siap: Laporan(id=1, prioritas=2)
Exception in thread "main"
    java.lang.IllegalStateException: Gagal memuat 7
        at FileKt.muat(File.kt:9)
        at FileKt.main(File.kt:15)
Caused by: java.lang.NumberFormatException:
    For input string: "tinggi"
        at java.lang.Integer.parseInt(Integer.java:662)
        at FileKt.keLaporan(File.kt:4)
        at FileKt.muat(File.kt:7)
        ... 2 more
```

Bagian	Artinya
Baris paling atas	Error terluar dan pesannya, sering hanya pembungkus
Caused by	Penyebab aslinya. Kalau ada beberapa, baca yang paling bawah
Baris pertama FileKt di bawahnya	Kode kita sendiri, tempat memperbaiki
java, android, kotlinx	Kode library yang ikut terseret, lewati dulu

Stack trace yang sama di Logcat

Membaca stack trace

```
Logcat package:mine level:ERROR
FATAL EXCEPTION: main
Process: id.ac.itera.laporkampus, PID: 8123
java.lang.IllegalStateException: Gagal memuat laporan 7
    at id.ac.itera.laporkampus.ui.DetailViewModel$muat$1
        .invokeSuspend(DetailViewModel.kt:34)
    at kotlinx.coroutines.DispatchedTask.run(DispatchedTask.kt:108)
    at android.os.Looper.loop(Looper.java:288)
Caused by: java.lang.NumberFormatException: For input string: "tinggi"
    at java.lang.Integer.parseInt(Integer.java:797)
    at id.ac.itera.laporkampus.data.LaporanDto.keLaporan(LaporanDto.kt:21)
    at id.ac.itera.laporkampus.data.SumberLaporanJaringan.ambil(...)
```

data/LaporanDto.kt

```
20 fun LaporanDto.keLaporan() = Laporan(
21     prioritas = prioritas.toInt(),
22     judul = judul, status = status.keStatus())
```

4 Buka berkas itu

Klik tautan di Logcat, Android Studio membuka barisnya.



Lewati baris java, android, dan kotlin. Itu kode library yang hanya ikut terseret.

Server mengirim prioritas "tinggi", bukan angka.

Perbaiki: toIntOrNull() dengan nilai bawaan.

COBA LANGSUNG



Scan atau klik, tekan Run, lalu perbaiki satu baris sampai ketiga laporan tercetak.

[Jalankan kode berbug di Kotlin Playground](#)

Sumber: Kotlin Playground, JetBrains. Kodenya sama dengan slide Kode yang akan crash.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Log yang mudah disaring

commonMain/ui/DetailViewModel.kt

```
fun muat(id: Int) {  
    println("LaporKampus muat id=$id")  
    viewModelScope.launch {  
        val hasil = sumber.ambil(id)  
        println("LaporKampus status=${hasil.status}")  
    }  
}
```

`println` dari `commonMain` masuk ke Logcat dengan tag `System.out`. Awalan yang sama di setiap pesan membuatnya mudah dicari.

Filter Logcat	Yang tampil
package:mine	Hanya aplikasi dari proyek yang terbuka
level:ERROR	Hanya error dan yang lebih parah
is:crash	Hanya crash aplikasi
message:LaporKampus	Pesan yang memuat kata itu

Bagian 5

Coba sendiri, lalu bawa ke proyekmu

Tiga latihan test dan Koin dengan observasi praktik, test untuk proyek kelompok, dan checkpoint proyek.

Yang saya amati selama latihan hari ini

Indikator RPS: **Test coverage minimal 60%, DI terkonfigurasi.** Diamati dari latihan 1 sampai 3, CounterTest, SessionLoggerTest, KoinModuleTest.

Kriteria	Sangat baik 85 sampai 100	Baik 70 sampai 84	Cukup 55 sampai 69	Kurang di bawah 55
Test dan DI CPMK0502, 4%	Coverage 60% atau lebih, Koin terkonfigurasi, dan bisa menjelaskan fungsi fake di test-nya	Coverage 40 sampai 59% dan Koin jalan	Ada test yang lulus, coverage di bawah 40% atau Koin belum jalan	Belum ada test yang jalan

Individu, bobot 4% nilai akhir. Bukti: push ke repository latihan pribadimu, lalu kirim tautan commit di LMS sebelum kelas selesai.

Tiga latihan di repo mata kuliah

Latihan 1

CounterTest.kt

Tiga test untuk increment, decrement di nol, dan reset dengan pola Arrange, Act, Assert.

Latihan 2

SessionLoggerTest.kt

FakeClockProvider dengan TestTimeSource, jam palsu yang hanya maju saat disuruh.

Latihan 3

KoinModuleTest.kt

startKoin dengan appModule, inject Counter dan SessionLogger, lalu stopKoin.

Semuanya di folder **P11 Testing dan DI Hands-on**, di `commonTest/.../latihan`. Jalankan dengan `./gradlew :composeApp:desktopTest`. Yang selesai duluan: tulis test bisaKirim dan FormLaporanViewModel di proyek kelompok.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
NotImplementedError: An operation is not implemented	TODO() di berkas latihan belum diganti	Lengkapi TODO, atau jalankan satu kelas saja
expected:<1> but was:<0>	Coroutine di viewModelScope belum jalan	<code>advanceUntilIdle()</code> sebelum assert
Test ViewModel kadang gagal, atau error Main dispatcher	viewModelScope memakai Dispatchers.Main asli	<code>Dispatchers.setMain</code> di BeforeTest
KoinAppAlreadyStartedException	startKoin dua kali tanpa stopKoin	<code>stopKoin()</code> di AfterTest
NoDefinitionFoundException	Modul berisi definisinya belum didaftarkan	Tambahkan modul itu di modules()
Test kadang lulus, kadang gagal	Bergantung pada waktu asli	Fake waktu seperti TestTimeSource

Test dan Koin untuk tema kelompokmu

1 Fake untuk setiap sumber
Sumber data dan fitur AI tema kalian punya versi uji yang bisa diatur gagal.

2 Sepuluh unit test
UiState dan ViewModel: aturan simpan, berhasil, gagal, dan memuat.

3 Tiga UI test
Form, daftar, dan layar AI dengan `runComposeUiTest`.

4 Koin
Modul data dan ui, layar memakai `koinViewModel`, modul uji untuk test.

CONTOH DARI LAPORKAMPUS

`FormLaporanUiStateTest` empat test, `FormLaporanViewModelTest` tiga, `AsistenViewModelTest` tiga, dan `FormLaporanUiTest` tiga. Semuanya lulus dengan `desktopTest` dalam hitungan detik.

Checkpoint proyek minggu 10

- 1 Koin terpasang**
Modul data dan ui, layar mengambil ViewModel dengan koinViewModel.
- 2 Sepuluh unit test**
UiState dan ViewModel diuji dengan fake, tanpa jaringan dan tanpa kunci API.
- 3 Tiga UI test**
Tiga layar diuji dengan runComposeUiTest, semuanya lulus.

Dikumpulkan

Push ke repository kelompok. README memuat perintah menjalankan test dan tangkapan layar desktopTest yang lulus.

Dicek di awal pertemuan berikutnya: tuntas, sebagian, atau belum.

TUTUP

Tiket keluar



- 1 Fake**
Apa itu fake, dan kenapa dipakai di unit test?
- 2 ViewModel**
Kenapa ViewModel lebih mudah diuji daripada composable?
- 3 Koin**
Apa yang berubah di layar saat dependensi disediakan Koin?

Jawab singkat di kertas atau formulir LMS sebelum keluar kelas.

TUTUP

Minggu depan: arsitektur dan CI



MASALAH MINGGU 11

Kode kelompok sudah besar dan dikerjakan bertiga. Bagaimana memastikan setiap push tetap bisa di-build dan semua test lulus, tanpa mengandalkan laptop salah satu anggota?

Persiapan

Pastikan desktopTest lulus di laptop setiap anggota. Bawa draf kebutuhan dan pembagian kerja kelompok untuk sprint 1.

Terima kasih

Sampai jumpa di pertemuan 11

Ada pertanyaan? Sampaikan sekarang, lewat LMS, atau email ke muhammad.algifari@if.itera.ac.id